

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №6
Дисциплина: Теория параллельных алгоритмов

Работу выполнил: _____ Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и
информационные технологии

Преподаватель: _____ Нигодин Е.А.

Краснодар
2026

Тема. Умножение матриц на CUDA.

Цель. Распараллелить процесс перемножения матриц с помощью CUDA.

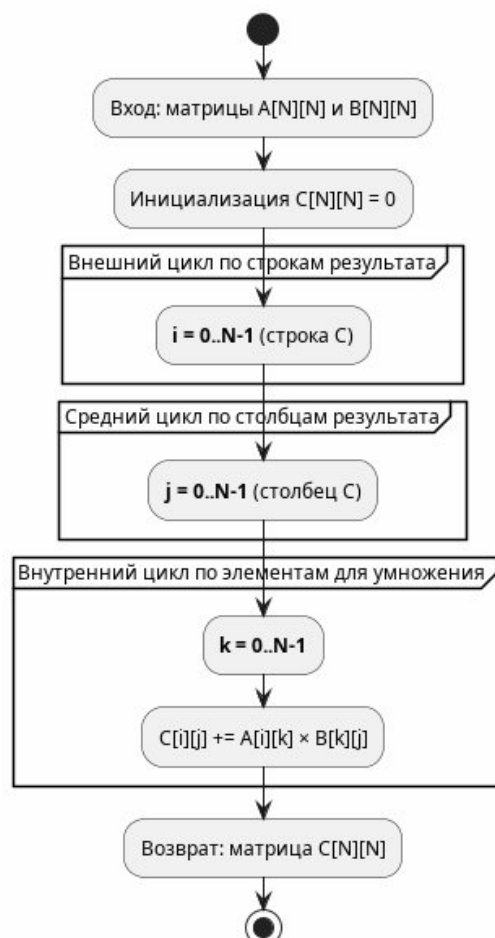
Задание. Составить последовательный алгоритм. Построить граф алгоритма. Исследовать на нём зависимости и возможности распараллеливания. Выполнить параллельную реализацию для случаев ленточного и блочного разделения матриц. Составить таблицу, отражающую сравнительное время выполнения последовательного и параллельных алгоритмов для разных размеров матриц.

Входные данные: матрица A размерности $n \times n$ и матрица B размерности $n \times n$. Необходимо получить матрицу C путём перемножения A на B. Элементы искомой матрицы C вычисляются следующим образом:

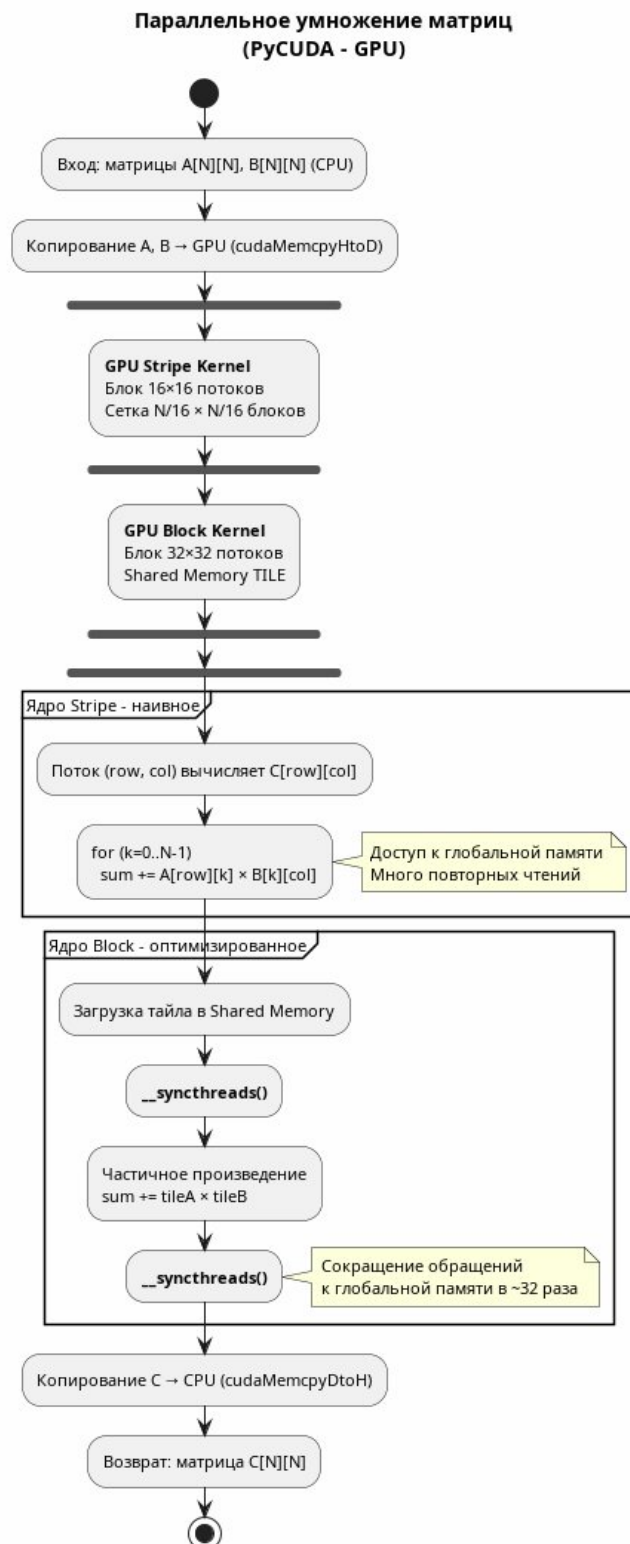
$$c_{ij} = \sum_{k=0}^{n-1} a_{ik} b_{kj} \quad 0 \leq i < n, 0 \leq j < n$$

Последовательный алгоритм перемножения можно схематично изобразить в следующем виде:

**Последовательное умножение матриц
(DOT Product - CPU)**



Возможность распараллеливания – представляем первую матрицу в виде нескольких последовательностей горизонтальных лент, каждая из которых параллельно перемножается со второй матрицей, после чего все результирующие ленты мы объединяем в одну искомую матрицу. Помимо рассмотрения матриц в виде наборов строк и столбцов, также можно использовать блочное представление матриц:



Реализуем параллельное ленточное умножение матриц:

```
def gpu_stripe_multiply(A, B, N):
    C = np.zeros((N, N), dtype=np.float32)

    A_gpu = cuda.mem_alloc(A.nbytes)
    B_gpu = cuda.mem_alloc(B.nbytes)
    C_gpu = cuda.mem_alloc(C.nbytes)

    cuda.memcpy_htod(A_gpu, A)
    cuda.memcpy_htod(B_gpu, B)

    block_size = (16, 16, 1)
    grid_size = ((N + block_size[0] - 1) // block_size[0],
                  (N + block_size[1] - 1) // block_size[1])

    kernel_stripe(A_gpu, B_gpu, C_gpu, np.int32(N),
                  block=block_size, grid=grid_size)

    cuda.memcpy_dtoh(C, C_gpu)
    cuda.Context.synchronize()

    return C
```

Напишем функцию для блочного перемножения матриц:

```
def gpu_block_multiply(A, B, N, TILE_SIZE=32):
    C = np.zeros((N, N), dtype=np.float32)

    A_gpu = cuda.mem_alloc(A.nbytes)
    B_gpu = cuda.mem_alloc(B.nbytes)
    C_gpu = cuda.mem_alloc(C.nbytes)

    cuda.memcpy_htod(A_gpu, A)
    cuda.memcpy_htod(B_gpu, B)

    block_size = (TILE_SIZE, TILE_SIZE, 1)
    grid_size = ((N + TILE_SIZE - 1) // TILE_SIZE,
                  (N + TILE_SIZE - 1) // TILE_SIZE)

    kernel_block(A_gpu, B_gpu, C_gpu, np.int32(N),
                 block=block_size, grid=grid_size)

    cuda.memcpy_dtoh(C, C_gpu)
    cuda.Context.synchronize()

    return C
```

Полный код итоговой программы:

```
import numpy as np
import pycuda.driver as cuda
from pycuda.compiler import SourceModule
import pycuda.autoinit
import time
import sys
import os

kernel_stripe_code = """
__global__ void matmul_stripe(float *A, float *B, float *C, int N)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < N && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < N; ++k) {
            sum += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}
"""

kernel_block_code = """
#define TILE_SIZE 32

__global__ void matmul_block(float *A, float *B, float *C, int N)
{
    __shared__ float tileA[TILE_SIZE][TILE_SIZE];
    __shared__ float tileB[TILE_SIZE][TILE_SIZE];

    int row = blockIdx.y * TILE_SIZE + threadIdx.y;
    int col = blockIdx.x * TILE_SIZE + threadIdx.x;

    float sum = 0.0f;

    for (int t = 0; t < (N + TILE_SIZE - 1) / TILE_SIZE; ++t) {
        int tiledRow = TILE_SIZE * t + threadIdx.y;
        int tiledCol = TILE_SIZE * t + threadIdx.x;

        if (row < N && tiledCol < N) {
            tileA[threadIdx.y][threadIdx.x] = A[row * N + tiledCol];
        } else {
            tileA[threadIdx.y][threadIdx.x] = 0.0f;
        }

        if (tiledRow < N && col < N) {
            tileB[threadIdx.y][threadIdx.x] = B[tiledRow * N + col];
        } else {
            tileB[threadIdx.y][threadIdx.x] = 0.0f;
        }

        __syncthreads();

        for (int k = 0; k < TILE_SIZE; ++k) {
            sum += tileA[threadIdx.y][k] * tileB[k][threadIdx.x];
        }

        __syncthreads();
    }

    if (row < N && col < N) {
        C[row * N + col] = sum;
    }
}
"""

compile_options = [
    '--allow-unsupported-compiler',
    '-D_ALLOW_COMPILER_AND_STL_VERSION_MISMATCH'
]
```

```

old_stderr = sys.stderr
sys.stderr = open(os.devnull, 'w')

mod_stripe = SourceModule(kernel_stripe_code, options=compile_options)
mod_block = SourceModule(kernel_block_code, options=compile_options)

sys.stderr = old_stderr

kernel_stripe = mod_stripe.get_function("matmul_stripe")
kernel_block = mod_block.get_function("matmul_block")

def sequential_multiply(A, B):
    return np.dot(A, B)

def gpu_stripe_multiply(A, B, N):
    C = np.zeros((N, N), dtype=np.float32)

    A_gpu = cuda.mem_alloc(A.nbytes)
    B_gpu = cuda.mem_alloc(B.nbytes)
    C_gpu = cuda.mem_alloc(C.nbytes)

    cuda.memcpy_htod(A_gpu, A)
    cuda.memcpy_htod(B_gpu, B)

    block_size = (16, 16, 1)
    grid_size = ((N + block_size[0] - 1) // block_size[0],
                  (N + block_size[1] - 1) // block_size[1])

    kernel_stripe(A_gpu, B_gpu, C_gpu, np.int32(N),
                  block=block_size, grid=grid_size)

    cuda.memcpy_dtoh(C, C_gpu)
    cuda.Context.synchronize()

    return C

def gpu_block_multiply(A, B, N, TILE_SIZE=32):
    C = np.zeros((N, N), dtype=np.float32)

    A_gpu = cuda.mem_alloc(A.nbytes)
    B_gpu = cuda.mem_alloc(B.nbytes)
    C_gpu = cuda.mem_alloc(C.nbytes)

    cuda.memcpy_htod(A_gpu, A)
    cuda.memcpy_htod(B_gpu, B)

    block_size = (TILE_SIZE, TILE_SIZE, 1)
    grid_size = ((N + TILE_SIZE - 1) // TILE_SIZE,
                  (N + TILE_SIZE - 1) // TILE_SIZE)

    kernel_block(A_gpu, B_gpu, C_gpu, np.int32(N),
                  block=block_size, grid=grid_size)

    cuda.memcpy_dtoh(C, C_gpu)
    cuda.Context.synchronize()

    return C

def run_benchmark():
    sizes = [256, 512, 1024, 2048]

    print("\n")
    print("=" * 105)
    print(f'{"Size":<10} {"Method":<20} {"Time (ms)":<15} {"Speedup":<10}')
    print("=" * 105)

    results = []

    for N in sizes:
        np.random.seed(42)
        A = np.random.randn(N, N).astype(np.float32)
        B = np.random.randn(N, N).astype(np.float32)

        start = time.perf_counter()
        C_seq = sequential_multiply(A, B)
        cpu_time = (time.perf_counter() - start) * 1000

```

```

print(f"{N:<10} {'Sequential (CPU)':<20} {cpu_time:<15.2f} {'1.00x':<10}")

results.append({
    'size': N,
    'method': 'Sequential',
    'time': cpu_time,
    'speedup': 1.0
})

start = time.perf_counter()
C_stripe = gpu_stripe_multiply(A, B, N)
stripe_time = (time.perf_counter() - start) * 1000
stripe_speedup = cpu_time / stripe_time
stripe_speedup_str = f"{stripe_speedup:.2f}x"

print(f"{N:<10} {'GPU Stripe':<20} {stripe_time:<15.2f} {stripe_speedup_str:<10}")

results.append({
    'size': N,
    'method': 'Stripe',
    'time': stripe_time,
    'speedup': stripe_speedup
})

start = time.perf_counter()
C_block = gpu_block_multiply(A, B, N)
block_time = (time.perf_counter() - start) * 1000
block_speedup = cpu_time / block_time
block_speedup_str = f"{block_speedup:.2f}x"

print(f"{N:<10} {'GPU Block':<20} {block_time:<15.2f} {block_speedup_str:<10}")

results.append({
    'size': N,
    'method': 'Block',
    'time': block_time,
    'speedup': block_speedup
})

diff_stripe = np.max(np.abs(C_seq - C_stripe))
diff_block = np.max(np.abs(C_seq - C_block))

if diff_stripe < 1e-3 and diff_block < 1e-3:
    print(f"{'':<10} {'Verification':<20} {'OK':<15}")
else:
    print(f"{'':<10} {'Verification':<20} {'FAIL':<15}")

print("-" * 105)

print("\n")
print("=" * 105)
print("SUMMARY TABLE")
print("=" * 105)
print(f"{'Size':<10} {'CPU (ms)':<15} {'GPU Stripe (ms)':<18} {'GPU Block (ms)':<17} {'Stripe Speedup':<16} {'Block Speedup':<15}")
print("-" * 105)

for N in sizes:
    size_results = [r for r in results if r['size'] == N]
    cpu = next(r for r in size_results if r['method'] == 'Sequential')
    stripe = next(r for r in size_results if r['method'] == 'Stripe')
    block = next(r for r in size_results if r['method'] == 'Block')

    stripe_spd = f"{stripe['speedup']:.2f}x"
    block_spd = f"{block['speedup']:.2f}x"

    print(f"{N:<10} {cpu['time']:<15.2f} {stripe['time']:<18.2f} {block['time']:<17.2f} "
          f"{stripe_spd:<16} {block_spd:<15}")

print("=" * 105)

return results

def test_tile_sizes():
    N = 1024
    np.random.seed(42)

```

```

A = np.random.randn(N, N).astype(np.float32)
B = np.random.randn(N, N).astype(np.float32)

start = time.perf_counter()
C_seq = sequential_multiply(A, B)
cpu_time = (time.perf_counter() - start) * 1000

print("\n")
print("=" * 60)
print(f"Tile Size IMPACT ON PERFORMANCE (N={N})")
print("=" * 60)
print(f"{'Tile Size':<15} {'Time (ms)':<15} {'Speedup':<15}")
print("-" * 60)

tile_sizes = [8, 16, 32]
for tile_size in tile_sizes:
    start = time.perf_counter()
    C_block = gpu_block_multiply(A, B, N, tile_size)
    block_time = (time.perf_counter() - start) * 1000
    speedup = cpu_time / block_time
    speedup_str = f"{speedup:.2f}x"

    print(f"{'tile_size':<15} {'block_time':<15.2f} {'speedup_str':<15}")

print("=" * 60)

def main():
    print("=" * 105)
    print("MATRIX MULTIPLICATION: SEQUENTIAL vs PARALLEL (PyCUDA)")
    print("=" * 105)

    device = cuda.Device(0)
    print(f"\nGPU Information:")
    print(f"  Device: {device.name()}")
    print(f"  Compute Capability: {device.compute_capability()}")
    print(f"  Total Memory: {device.total_memory() / 1024**3:.2f} GB")
    print(f"  Max Threads per Block: {device.max_threads_per_block}")
    print(f"  Max Block Dimensions: {device.max_block_dim_x} x {device.max_block_dim_y} x {device.max_block_dim_z}")

    results = run_benchmark()

    test_tile_sizes()

if __name__ == "__main__":
    main()

```


Зафиксируем показатели работы алгоритмов:

MATRIX MULTIPLICATION: SEQUENTIAL vs PARALLEL (PyCUDA)					
GPU Information:					
Device: NVIDIA GeForce RTX 4070 Laptop GPU					
Compute Capability: (8, 9)					
Total Memory: 8.00 GB					
Max Threads per Block: 1024					
Max Block Dimensions: 1024 x 1024 x 64					
Size	Method	Time (ms)	Speedup		
256	Sequential (CPU)	2.59	1.00x		
256	GPU Stripe	1.18	2.18x		
256	GPU Block	0.58	4.43x		
512	Sequential (CPU)	1.30	1.00x		
512	GPU Stripe	1.27	1.02x		
512	GPU Block	1.13	1.15x		
1024	Sequential (CPU)	5.86	1.00x		
1024	GPU Stripe	5.50	1.07x		
1024	GPU Block	4.23	1.38x		
2048	Sequential (CPU)	44.15	1.00x		
2048	GPU Stripe	25.94	1.70x		
2048	GPU Block	21.41	2.06x		
SUMMARY TABLE					
Size	CPU (ms)	GPU Stripe (ms)	GPU Block (ms)	Stripe Speedup	Block Speedup
256	2.59	1.18	0.58	2.18x	4.43x
512	1.30	1.27	1.13	1.02x	1.15x
1024	5.86	5.50	4.23	1.07x	1.38x
2048	44.15	25.94	21.41	1.70x	2.06x

Вывод. Мы рассмотрели процесс перемножения матриц на CUDA, изобразили его схематично, после чего, изучив возможности распараллеливания, выполнили ленточное и блочное перемножение матриц с замером времени для последовательного и параллельного случаев.